

PowerShell コマンドが見つからない問題

<https://gemini.google.com/app/9064b28601f13a59>

User prompt: そもそもPiへのさぎょうはすべてデスクトップ側で行っていました

Response: あー、納得です！！完全に点が線でつながりました！

元々の開発フローは「デスクトップPC側で Next.js をビルドして Pi にデプロイ（転送）していた」のですね。だからデスクトップPCには Node.js/npm やビルド環境が揃っていて、今使っているノートPCには npm が入っていなかったわけです。

それなら話はめちゃくちゃシンプルです！わざわざノートPCの環境を構築し直さなくても、「作業をいつものデスクトップPC側から行う」のが一番自然で確実です。

デスクトップPC側でやること（1分で終わります）

デスクトップPCの PowerShell（または Terminal）を開いて、以下を実行するだけです。

1. リポジトリの場所へ移動

```
cd C:\Users\%ssliv%\OneDrive\Desktop\Myproject\HomeLLM
```

(※ パスはデスクトップPC上の配置に合わせて変更してください)

2. 接続先APIを「ノートPCの Tailscale IP」に指定してデプロイ

.env.local の接続先をノートPCの Tailscale IP (100.104.157.27) にして Pi へ転送します。

```
# web ディレクトリへ移動
cd services\web

# ノートPCの Tailscale IP (100.104.157.27) を指定
"@
NEXT_PUBLIC_JOB_HUNTING_API_URL=http://100.104.157.27:8000
NEXT_PUBLIC_VIDEO_INGEST_API_URL=http://100.104.157.27:8090
"@ | Set-Content .\env.local -Encoding utf8

# ルートに戻って Pi へデプロイ実行
cd ..
powershell -File .\scripts\deploy-web-pi.ps1
```

仕上げ（ノートPC側）

ノートPC側では、API のポートフォワード (kubectl port-forward --address 0.0.0.0 svc/job-hunting-api 8000:8000) を開いたままにしておいてください。

デスクトップからのデプロイが終わったら、ブラウザ (http://usudonsdev.local/) で **Refresh** を押せば、接続先が http://100.104.157.27:8000/ に更新されて見事に **UP (緑色)** に切り替わります！

User prompt: SSHするコマンドが一切ないけどこれでどうやってPiにアクセスするの

Response: その疑問、鋭いです！「SSHコマンドを実行していないのに、どうやってPiに転送するんだ？」と思いますよね。

実は、先ほど作成した .local\pi-deploy.env とスクリプト (deploy-web-pi.ps1) の仕組みの中に理由があります。

スクリプトの裏側でやっていること

deploy-web-pi.ps1 は単なるコマンドの自動化スクリプトで、内部で以下のような処理を順番に行っています。

1. **PC側でビルド**：npm run build を実行し、画面のプログラム (HTML/JS) を生成する。
2. **.local\pi-deploy.env を読み込む**：設定された PI_SSH=usudonsdev@100.101.237.17 や PI_WEB_ROOT=/var/www/homellm を読み取る。

3. 裏で SSH 転送を実行：スクリプト内部の処理として、scp や rsync（内部的に SSH 接続を使うファイル転送コマンド）を自動的に呼び出し、生成されたファイルを Pi の /var/www/homellm に流し込む。

つまり、手動で ssh usudonsdev@... と打たなくても、スクリプトが設定ファイルの情報を使って裏で自動的に Pi へ SSH 接続してファイルを書き換えてくれているという仕組みです。

デスクトップPCで実行する際のご注意

デスクトップPCからデプロイする際も、スクリプトが Pi の宛先を知る必要があるため、デスクトップPC上のプロジェクトにも .local¥pi-deploy.env が配置されている必要があります。

もしデスクトップPC側でスクリプトを実行して「Missing .local¥pi-deploy.env」と出た場合は、デスクトップPC側の PowerShell で以下を実行してからデプロイしてくださいね。

```
New-Item -ItemType Directory -Force -Path .¥.local | Out-Null
```

```
@”
PI_SSH=usudonsdev@100.101.237.17
PI_WEB_ROOT=/var/www/homellm
"@ | Set-Content .¥.local¥pi-deploy.env -Encoding utf8
```

User prompt: デスク側です PS C:\Users\ssliv\Documents\GitHub\HomeLLM> PS
C:\Users\ssliv\Documents\GitHub\HomeLLM> # web ディレクトリへ移動 >> cd services\web >> >> # ノートPCの Tailscale IP (100.104.157.27) を指定 >> @">> NEXT_PUBLIC_JOB_HUNTING_API_URL=http://100.104.157.27:8000 >>
NEXT_PUBLIC_VIDEO_INGEST_API_URL=http://100.104.157.27:8090 >> "@ | Set-Content .¥.env.local -Encoding utf8 >> >> # ルートに戻って Pi ヘデプロイ実行 >> cd .. >> powershell -File .¥scripts\deploy-web-pi.ps1 -File パラメーターの引数 '.¥scripts\deploy-web-pi.ps1' は存在しません。既存の '.ps1' ファイルのパスを -File パラメーターの引数として指定してください。 Windows PowerShell Copyright (C) Microsoft Corporation. All rights reserved. PS
C:\Users\ssliv\Documents\GitHub\HomeLLM\services>

Response: エラーの原因はシンプルです！

cd .. で戻る位置がずれちゃいました。元々 C:¥Users¥ssliv¥Documents¥GitHub¥HomeLLM にいた状態で services¥web まで2階層入ったので、cd .. を1回打っただけだと HomeLLM¥services に留まってしまっています。

プロジェクト直下（HomeLLM）に戻ってから実行すれば通ります！

デスクトップPCの今の画面（services ディレクトリ）で、そのまま以下を打ち込んでみてください！

```
# もう1つ上の HomeLLM 直下に戻る
cd ..
```

```
# デプロイ実行
powershell -File .¥scripts¥deploy-web-pi.ps1
```

もし Missing .local¥pi-deploy.env と出たら、先ほどと同じように以下を流し込んでから再実行してください。

```
New-Item -ItemType Directory -Force -Path .¥.local | Out-Null
@"
PI_SSH=usudonsdev@100.101.237.17
PI_WEB_ROOT=/var/www/homellm
"@ | Set-Content .¥.local¥pi-deploy.env -Encoding utf8
```

```
powershell -File .¥scripts¥deploy-web-pi.ps1
```

ビルドが始まって Pi への転送まで完了したら、ブラウザの **Status ページ（Refresh）** を確認してみましょう！

User prompt: デスク側です PS C:\Users\ssliv\Documents\GitHub\HomeLLM> PS
C:\Users\ssliv\Documents\GitHub\HomeLLM> # web ディレクトリへ移動 >> cd services\web >> >> # ノートPCの Tailscale IP (100.104.157.27) を指定 >> @">> NEXT_PUBLIC_JOB_HUNTING_API_URL=http://100.104.157.27:8000 >>
NEXT_PUBLIC_VIDEO_INGEST_API_URL=http://100.104.157.27:8090 >> "@ | Set-Content .¥.env.local -Encoding utf8 >> >> # ルートに戻って Pi ヘデプロイ実行 >> cd .. >> powershell -File .¥scripts\deploy-web-pi.ps1 -File パラメーターの引数 '.¥scripts\deploy-web-pi.ps1' は存在しません。既存の '.ps1' ファイルのパスを -File パラメーターの引数として指定してくださ

い。 Windows PowerShell Copyright (C) Microsoft Corporation. All rights reserved. PS
 C:\Users\ssliv\Documents\GitHub\HomeLLM\services> # もう1つ上の HomeLLM 直下に戻る >> cd .. >> >> # デプロイ実行 >>
 powershell -File .\scripts\deploy-web-pi.ps1 Building static export... > homellm-web@0.1.0 build > next build ▲ Next.js
 15.5.9 - Environments: .env.local Creating an optimized production build ... ✓ Compiled successfully in 2.2s ✓ Linting
 and checking validity of types ✓ Collecting page data ✓ Generating static pages (7/7) ✓ Collecting build traces ✓
 Exporting (2/2) ✓ Finalizing page optimization Route (app) Size First Load JS 1.1
 kB 103 kB 103 kB /_not-found 993 B 103 kB /experiences 1.58 kB 104 kB /rag
 1.34 kB 104 kB /videos 1.36 kB 104 kB + First Load JS shared by all 102 kB
 chunks/255-bc8b3b9a93d25a64.js 46 kB chunks/4bd1b696-c023c6e3521b1417.js 54.2 kB other shared chunks
 (total) 1.97 kB (Static) prerendered as static content Ensuring remote directory /var/www/homellm ...
 usudonsdev@usudonsdev.local's password:

Response: おおっ、ビルド大成功です！ばっちり静的ファイル（out）の出力まで完了しましたね！

今はスクリプトが自動で Pi 宛てにファイルを転送しようとして、**パスワードの入力待ち**になっています。

今やること

1. ターミナルで usudonsdev@usudonsdev.local's password: と聞かれているので、**Pi (usudonsdev ユーザー) の SSH パスワード**を入力して Enter を押してください。(※ 入力中、画面には文字や*が表示されませんが、内部ではちゃんと入力されているので気にせずそのまま打って Enter で大丈夫です！)
2. パスワード転送処理が 2〜3 回聞かれる場合があります（ディレクトリ作成用とファイル転送用）。その都度同じパスワードを入力してください。

転送が終わってプロンプト（PS C:\>）に戻ったらデプロイ完了です！

完了したら、ノートPCまたはブラウザで <http://usudonsdev.local/>（または Pi の IP）を開いて **Refresh** を押してみてください。

表示が http://100.104.157.27:8000 に切り替わり、緑色の **UP** になっているはずです！

User prompt: Status Production UI on the Pi. This page probes Windows APIs over Tailscale (or local port-forward for desktop demos). job-hunting API http://100.104.157.27:8000 DOWN Failed to fetch video-ingest API
 http://100.104.157.27:8090 DOWN Failed to fetch Refresh Pi hosts UI only. APIs and media stay on the Windows compute node.

Response: URL が http://100.104.157.27:8000（ノートPCの Tailscale IP）に切り替わりましたね！デプロイは大成功です！

ですが、まだ DOWN (Failed to fetch) になっている理由は、「**ノートPC (Windows) のファイアウォール**」が外部 (Pi) からの 8000 番ポートへの通信をブロックしているためです。

ノートPC (100.104.157.27) 側で以下の **2点** をサクッと通してあげれば、一気に **UP (緑色)** に変わります！

1. ノートPCのファイアウォールで 8000 番ポートを開放

ノートPCの **管理者権限で開いた PowerShell** で、以下のコマンドをそのまま実行してください。

```
New-NetFirewallRule -DisplayName "HomeLLM-API-8000" -Direction Inbound -LocalPort 8000 -Protocol TCP -Action Allow
```

(もし video-ingest 用に 8090 も開けるなら 8000, 8090 にしてください)

2. ノートPCの port-forward が動いているか確認

ノートPCの通常の PowerShell 窓で、以下が実行されたままであることを確認します。

```
kubectrl -n job-hunting port-forward --address 0.0.0.0 svc/job-hunting-api 8000:8000
```

 **最終確認！**

ファイアウォール許可が完了したら、ブラウザで「**Refresh**」ボタンを押してみてください！

これで Pi（ブラウザ）→Tailscale ネットワーク→ノートPCの 8000 番ポート（Kubernetes サービス）への通信がつながり、念願の **UP（緑色）** に切り替わるはずです！

User prompt: HomeLLM Status Experiences RAG Valorant Status Production UI on the Pi. This page probes Windows APIs over Tailscale (or local port-forward for desktop demos). job-hunting API http://100.104.157.27:8000 DOWN Failed to fetch video-ingest API http://100.104.157.27:8090 DOWN checking... Refresh Pi hosts UI only. APIs and media stay on the Windows compute node. ノータイムですぐDownって表示されてる

Response: Failed to fetchがノータイム（即座）で出るのは、タイムアウトによる通信エラーではなく、**ブラウザが CORS（Cross-Origin Resource Sharing）でリクエストを弾いているか、Windows 側のポートがまだ空いていない（またはプロセスがバインドされていない）** ときに起きる典型的な挙動です！

原因を切り分けるために、ノートPCで**2つ**だけ確認しましょう。

Check 1: そもそもノートPC自身から叩けるか？

ノートPCの PowerShell で以下を実行してみてください。

```
curl.exe -s http://100.104.157.27:8000/health
```

- これで **JSON が返ってくる場合** 🟡 API は生きています！原因は **Windows ファイアウォール** か **CORS（ブラウザ制限）** です。
- これで**エラーになる場合** 🟡 port-forward が落ちているか、127.0.0.1 でしか待受けていません。

Check 2: port-forward の再起動（0.0.0.0 明示）

もし Check 1 でエラーになった場合は、ノートPCで port-forward を一度 Ctrl+C で止め、以下を再実行してください。

```
kubectl -n job-hunting port-forward --address 0.0.0.0 svc/job-hunting-api 8000:8000
```

Check 1 で JSON が返ってきた場合の対処法

ブラウザ（Mixed Content 規制や CORS）の問題か、Windows ファイアウォールです。管理者 PowerShell で以下を実行してファイアウォールを確実に開けてみてください。

```
New-NetFirewallRule -DisplayName "HomeLLM-API" -Direction Inbound -LocalPort 8000,8090 -Protocol TCP -Action Allow
```

まずは Check 1 の curl.exe の結果がどうなるか教えてください！

User prompt: ノート側です PS C:\Users\ssliv\OneDrive\Desktop\Myproject\HomeLLM> curl.exe -s http://100.104.157.27:8000/health PS C:\Users\ssliv\OneDrive\Desktop\Myproject\HomeLLM> curl.exe -s http://100.104.157.27:8000/health PS C:\Users\ssliv\OneDrive\Desktop\Myproject\HomeLLM> kubectl -n job-hunting port-forward --address 0.0.0.0 svc/job-hunting-api 8000:8000 Unable to listen on port 8000: Listeners failed to create with the following errors: [unable to create listener: Error listen tcp4 0.0.0.0:8000: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.] error: unable to listen on any of the requested ports: [{8000 8000}] PS C:\Users\ssliv\OneDrive\Desktop\Myproject\HomeLLM> New-NetFirewallRule -DisplayName "HomeLLM-API" -Direction Inbound -LocalPort 8000,8090 -Protocol TCP -Action Allow Name : {575ea942-eef9-44ac-891a-9fffe83d7d72} DisplayName : HomeLLM-API Description : DisplayGroup : Group : Enabled : True Profile : Any Platform : {} Direction : Inbound Action : Allow EdgeTraversalPolicy : Block LooseSourceMapping : False LocalOnlyMapping : False Owner : PrimaryStatus : OK Status : 規則は、ストアから正常に解析されました。(65536) EnforcementStatus : NotApplicable PolicyStoreSource : PersistentStore PolicyStoreSourceType : Local RemoteDynamicKeywordAddresses : {} PolicyAppId : PackageFamilyName :

Response: ログの実行結果から、状況が完全にピンポイントで特定できました！

1. curl.exe が何も返さない (空欄)

2. port-forward が bind: Only one usage... で失敗する

これは、「port-forward が裏 (別窓や別プロセス) ですでに動いているけれど、127.0.0.1 (ローカルホスト専用) でしか開いていない」状態です。127.0.0.1 で専有されているため、Tailscale IP (100.104.157.27) 宛でのリクエストを拾えず、ノートタイムで弾かれていました。

一度その古い port-forward を強制的に終了 (kill) して、--address 0.0.0.0 で掴み直せば一発で解決します！

ノートPCで実行する手順 (3ステップ)

ノートPCの PowerShell で、以下の通り順番に実行してみてください。

1. 古い kubectl (port-forward) プロセスを一度全部殺す

```
Stop-Process -Name "kubectl" -Force -ErrorAction SilentlyContinue
```

2. 改めて --address 0.0.0.0 で port-forward を起動 (この窓は開けたまま！)

```
kubectl -n job-hunting port-forward --address 0.0.0.0 svc/job-hunting-api 8000:8000
```

(※ 今度は *Unable to listen...* のエラーが出ず、*Forwarding from 0.0.0.0:8000 -> 8000* と表示されて止まるはずです！)

3. 疎通テスト (別窓の PowerShell で)

```
curl.exe http://100.104.157.27:8000/health
```

これで {"status": "ok"} のようなレスポンスが返ってくれば勝利確定です！

そのままブラウザ (http://usudonsdev.local/) で **「Refresh」** を押してみてください！緑色の **UP** に切り替わり、画面が一気に動き出します！

User prompt: そもそも一瞬たりとも *Fetching* にならないのはフロント側に問題があるんじゃないの？

Response: おっしゃる通りです！一瞬も **「checking... / Fetching...」** にならず **ノータイムで DOWN** または **Failed to fetch** になるのは、フロントエンド (JavaScript / Next.js) 側の実装やブラウザの挙動による影響が極めて大きいです。

特に Next.js のフロントエンドでこの現象が起きる原因は、大きく分けて **3つ** あります。

原因 1. ブラウザの CORS / Mixed Content (セキュリティブロック)

ブラウザ (Chrome 等) は、Origin (呼び出し元) と Destination (呼び出し先) のプロトコルや IP/ドメインが異なる場合、実際に HTTP リクエストを送信する前 (Preflight / CORS チェック) の段階でブラウザ自身が即座にエラー判定を出します。

- **画面の URL:** [http://usudonsdev.local/](http://usudonsdev.local) または http://100.101.237.17
- **API の URL:** [http://100.104.157.27:8000/](http://100.104.157.27:8000)

ネットワーク層でパケットが飛ぶ前にブラウザ内で即死 (TypeError: Failed to fetch) するため、通信待ち時間が発生せずノータイムでエラーになります。

原因 2. fetch のキャッチ処理 (即時エラーハンドリング)

フロントのコード (Status ページのコンポーネント) で、fetch() が例外を吐いた瞬間に try-catch や .catch() で拾って、ステータスを即座に 'DOWN' に更新する実装になっている場合です。ブラウザが接続拒否 (Refused) や CORS 違反を検知すると fetch は 0 秒で reject されるため、UI 上のローディング状態 (checking...) が一瞬すぎて目視できずに DOWN に固定されます。

原因 3. クライアントサイドでの URL 解析エラー・環境変数のミスマッチ

Next.js 13+ (App Router 等) では、NEXT_PUBLIC_* がクライアント（ブラウザ）側で正しく評価されていないか、あるいは API のエンドポイントパス（/health や /api/v1/health など）がフロント側のコードで期待しているパスとズレているケースがあります。

フロント側の挙動を確定させる確認方法（デバッグ）

本当にフロント（ブラウザ）で何が起きているかを確認するには、ブラウザのデベロッパーツールを開くのが一番確実です。

1. <http://usudonsdev.local/> を開く。
2. F12 キー（または右クリック → 検証）を押してデベロッパーツールを開く。
3. 「Console (コンソール)」タブと「Network (ネットワーク)」タブを開いた状態で、画面の Refresh ボタンを押す。

画面で確認できること

- **Console タブに赤字で:** Access to fetch at 'http://100.104.157.27:8000/...' from origin 'http://usudonsdev.local/' has been blocked by CORS policy 🚩 **CORS エラー**（バックエンド側の CORS 許可設定が必要）
- **Network タブでステータスが:** (failed) net::ERR_CONNECTION_REFUSED 🚩 **Windows (ノートPC) 側でポートが閉じている、または port-forward が括めていない**
- **Network タブに 404 Not Found が出ている:** 🚩 フロントが叩いているパス（例: /health なのか /api/health なのか）が間違っている

一度 F12 の Console や Network にどんな赤字が出ているか見てみてください！一発で「フロントが何を理由に即死しているか」が判明します。

```
User prompt: (インデックス) :1 Access to fetch at 'http://100.104.157.27:8000/health' from origin
'http://usudonsdev.local' has been blocked by CORS policy: The request client is not a secure context and the resource is in
more-private address space `local`. 100.104.157.27:8000/health:1 Failed to load resource: net::ERR_FAILED (インデックス) :1 Access to fetch at
'http://100.104.157.27:8000/health' from origin 'http://usudonsdev.local' has been blocked by
CORS policy: The request client is not a secure context and the resource is in more-private address space `local`.
100.104.157.27:8000/health:1 Failed to load resource: net::ERR_FAILED (インデックス) :1 Access to fetch at
'http://100.104.157.27:8000/health' from origin 'http://usudonsdev.local' has been blocked by CORS policy: The request
client is not a secure context and the resource is in more-private address space `local`. 100.104.157.27:8000/health:1
Failed to load resource: net::ERR_FAILED (インデックス) :1 Access to fetch at 'http://100.104.157.27:8000/health' from
origin 'http://usudonsdev.local' has been blocked by CORS policy: The request client is not a secure context and the
resource is in more-private address space `local`. 100.104.157.27:8000/health:1 Failed to load resource: net::ERR_FAILED
100.104.157.27:8090/health:1 Failed to load resource: net::ERR_CONNECTION_TIMED_OUT 100.104.157.27:8090/health:1
Failed to load resource: net::ERR_CONNECTION_TIMED_OUT 100.104.157.27:8090/health:1 Failed to load resource:
net::ERR_CONNECTION_TIMED_OUT 100.104.157.27:8090/health:1 Failed to load resource:
net::ERR_CONNECTION_TIMED_OUT
```

Response: まさに原因が完璧に特定できました！コンソールログを出していただいて大正解です。

原因は **Chrome (Chromium) のセキュリティ仕様「Private Network Access (PNA)」** です。

エラーの正体

The request client is not a secure context and the resource is in more-private address space local.

- **何が起きているか:**
 - **送信元 (フロント) :** http://usudonsdev.local/ (非暗号化 HTTP = 非 Secure Context)
 - **宛先 (API) :** http://100.104.157.27:8000/ (プライベート IP アドレス空間)

- Chrome はセキュリティ上、「HTTP（非暗号化）のWebページから、ローカル/プライベートIPのAPIを勝手に叩くこと」を **Preflight（事前確認）の段階でノータイムで即座に拒否** します。これが一瞬で DOWN になっていた犯人です！

解決策（今すぐ試せる2つの方法）

開発中・検証中であれば、**方法 A（Chromeの設定を1つ変えるだけ）** が圧倒的に早いです！

方法 A: Chrome の Private Network Access 規制を一時的にオフにする（一番楽）

1. Chrome のアドレスバーに以下を入力して開きます。
`chrome://flags/#block-insecure-private-network-requests`
2. 「**Block insecure private network requests**」の項目を Disabled に変更します。
3. 画面右下に表示される「**Relaunch**」ボタンを押して Chrome を再起動します。
4. 再び `[http://usudonsdev.local](http://usudonsdev.local)` を開いて **Refresh** を押してみてください！

方法 B: Tailscale IP（IPアドレス直打ち）でフロントを開く

ドメイン名（usudonsdev.local）ではなく、Pi の Tailscale IP（100.101.237.17）経由でアクセスすると、ブラウザが「同じプライベートネットワーク同士の通信」と判定して規制を回避できる場合があります。

- **アクセス先:** `[http://100.101.237.17](http://100.101.237.17)`

まず方法 A を試してみてください。即座に CORS エラーが消えて **UP（緑色）** に切り替わるはずです！

User prompt: Insecure origins treated as secureTreat given (insecure) origins as secure origins. Multiple origins can be supplied as a comma-separated list. Origins must have their protocol specified e.g. "http://example.com". For the definition of secure contexts, see <https://w3c.github.io/webappsec-secure-contexts/> – Mac, Windows, Linux, ChromeOS, Android kore?

Response: それではなく、`#block-insecure-private-network-requests` というフラグです！

見つからない場合は、検索窓に `private network` と入力してみてください。

もし Chrome の最新バージョンなどでそのフラグ自体が消えている（非推奨になった）場合は、**今見つけてくれたそのフラグ（Insecure origins treated as secure）** を使って回避するのが正解です！

見つけてくれたフラグを使う場合の対処法

見つけてくれた項目（Insecure origins treated as secure）の入力欄に、以下のように **Pi の URL** を入力してください。

1. **入力欄に指定する値:**
`http://usudonsdev.local,http://100.101.237.17`
2. 右側のドロップダウンを **Enabled** に変更します。
3. 右下の「**Relaunch**」（再起動）ボタンを押します。

これで Chrome が `[http://usudonsdev.local](http://usudonsdev.local)` を「安全なオリジン（Secure Context）」として扱ってくれるようになるため、プライベート IP（`[http://100.104.157.27:8000](http://100.104.157.27:8000)`）への fetch ブロックが解除されます！

再起動後にページを開いて **Refresh** を押してみてください！

User prompt: job-hunting API `http://100.104.157.27:8000 UP job-hunting-api / model qwen3.5:9b`

Response: UP 表示しましたね！接続成功おめでとうございます！🎉

フロントエンド（Pi 上の Next.js）からノートPCの Kubernetes 上で動く job-hunting-api（qwen3.5:9b）へ無事にリクエストが到達し、ステータスが正常に取得できるようになりました。

ブラウザの PNA（Private Network Access）規制を許可したことで、http://usudonsdev.local（非 Secure Context）から Tailscale 内のプライベート IP（http://100.104.157.27:8000）への通信がブロックされずに通るようになったのが勝因ですね。

今回のトラブルシューティングのまとめ

1. **原因1（IP不一致）**：Pi 上のフロントが古いビルド（デスクトップの IP 100.125.196.76）を参照していた → **デスクトップからノートPCの IP 100.104.157.27 を埋め込んで再ビルド&デプロイ**
2. **原因2（ポート未解放/バインドミス）**：`kubectl port-forward` が 127.0.0.1 で専有されていた → **プロセスを kill して --address 0.0.0.0 で再起動**
3. **原因3（ブラウザのセキュリティブロック）**：Chrome の PNA 規制でノータイムで即死（CORS エラー） → **Insecure origins treated as secure で Pi のオリジンを許可**

これで Production UI からノートPC上の LLM / API を自由に叩ける環境が整いました！引き続き UI 側の機能テストや video-ingest 側の調整など、進めたい作業があればいつでも教えてください！
